Agentic AI Evaluation – Hyperscaler Comparison (MSFT, AWS, GCP)

1. Objective

Evaluate hyperscaler platforms (Microsoft, AWS, GCP) for building a scalable, enterprise-grade multi-agent system that supports:

- Semantic reasoning and planning
- Multi-layer agent orchestration
- Tool (MCP) ecosystem governance
- Observability, identity, and cost control
- Enterprise-grade lifecycle management (versioning, tenancy, prompt mgmt)

---

2. Business Use Case

User Query

Across my portfolio, identify products where price increases over the past 12 weeks drove buyer attrition; products that lost households, not just volume. Then build an audience of its lapsed buyers, scale it, and size it for activation.

Expected Outcome

The system should:

- Identify worst-performing product(s) across my portfolio
- Size the audience
- Provide follow-up action activation
- Activation action will be dummy

---

3. Target Architecture

3.1 Supervisor Agent

Responsibilities

- Interpret user intent through schemantic agent
- Decompose into execution plan
- Choose agents and sequence
- Pass minimal, relevant context
- Aggregate outputs and synthesize results
- Generate follow-up actions

Example Execution Plan

1. Invoke Scoping Agent

2. Invoke Pricing Opportunities Agent
3. Invoke Sizing Agent

3.2 Solution Orchestration Layer

Agents

- Liquid Activate Agent
- Pricing Agent
- Survey Agent (Dummy)
- Analytics Agent (Dummy)
- Brain Wave Agent (Dummy)
- Synapse Agent (Dummy)

Responsibilities

- Coordinate workflows across specialized agents
- Apply domain logic
- Handle dependencies and retries
- Potentially call MCP tools directly (in some scenarios)
- Dummy solution agents return hardcoded response. They are to demonstrate the capability of symantic agent that interprets the user intent.

3.3 Low-Level Specialized Agents

Core Agents

Scoping Agent (Liquid Activate Solution)

- Defines product/category (carbonated drinks)
- Defines time range
- Defines measures (sales, trips, etc.)

Sizing Agent (Liquid Activate Solution)

- Builds audience segments
- Calculates size of “lapsed buyers > 2 trips”

Activate Agent (Liquid Activate Solution)

- Dummy agent that returns a standard response.

Pricing Opportunities Agent (Pricing Solution)

- Identifies worst-performing products

Price Optimization Agent (Pricing Solution)

- Simulates pricing scenarios (e.g., +$0.30 impact)

3.4 MCP Tools Layer with gateway

Definition

Reusable tools/services used by agents:

- Data access (IRI/Circana datasets)
- Metrics/semantic calculations
- Query engines
- Liquid Activation API

Requirements

- Central tool registry
- Discoverable across teams
- Versioned and reusable
- Avoid duplication

---

4. Evaluation Dimensions

4.1 Agent Creation & Framework

Frameworks

- MSFT: Foundary
- AWS: Bedrock Agents
- GCP: Vertex AI Agents

Evaluate

- Multi-agent orchestration support
- Planning + tool usage
- Agent-to-agent communication using A2A protocol
- Implement a retry mechanism to handle failures in long-running workflows.
- Long-running agent workflows that operate in the background and notify the user upon completion, allowing them to continue with other tasks without waiting.

4.2 Workflow Creation & Management

Workflow Types

- Sequential — agents execute in a defined order, passing output from one step to the next
- Parallel — independent agents or branches run concurrently and results are merged
- Conditional — routing logic selects the next step based on runtime state or agent output (similar to LangGraph)

Evaluate

- Support for composing multi-step agent workflows with sequential, parallel, and conditional patterns
- Visual or programmatic workflow authoring (design-time vs runtime)
- Workflow versioning, validation, and safe rollout alongside agent changes

Key Questions

- Can workflows be defined and externalized as JSON (or similar) configuration rather than hard-coded in application logic?
- Does the SDK or framework load, validate, and execute workflow definitions from that external config?
- How are workflow steps bound to registered agents and tools at runtime?
- Can conditional edges and branching rules be expressed declaratively in the configuration?

4.3 Agent Registration & Discovery

- Central agent registry
- Skill/capability tagging
- Runtime discoverability

Key Questions

- How are agents published and versioned?
- Can supervisor dynamically discover agents?
- Is there metadata-driven discovery?
- How can we register agents that are built using third-party open-source tools or hyperscaler frameworks and hosted within our network?

4.4 Managing 100+ Enterprise Agents

- Registry design
- Skill taxonomy
- Ownership & governance

Best Practices

- Domain-based grouping (Pricing, Activation, Audience)
- Skill tagging (e.g., segmentation, pricing simulation)
- Reusable agent contracts

4.5 Observability & Traceability

What to Trace

- Agent decision steps
- Tool calls
- Execution flow (plan → agents → tools)
- Latency and cost

What NOT to Over-Trace

- Full prompts (cost + privacy risk)
- Large datasets passed between agents

Evaluate

- Built-in tracing tools

- Debugging support
- Cost observability
- Built in Evaluation / Experimentation tools

4.6 MCP Tools Architecture

Evaluate

- Tool deployment (API/function-based)
- Central tool registry
- Discoverability by agents

Key Questions

- How do multiple teams publish tools?
- How is duplication avoided?
- Are contracts standardized?
- Do you have any tool that helps agents navigate efficiently to the right mcp tool while minimizing the amount of context required?

4.7 Memory (Short-term & Long-term)

Short-Term

- Session/context memory within workflow

Long-Term

- Persistent knowledge
- Embeddings/vector stores

Evaluate

- Native vs external memory
- Retrieval mechanisms
- Context injection strategies
- Compacting and Dreaming
- Saving user preferences and selected favorites from interaction history.
- Session management across agents

4.8 Dynamic Model Selection

Requirement

- Planning → reasoning model
- Data ops → cheaper model
- Model availability by region

Evaluate

- Built-in routing capability

- Policy-based or agent-driven selection
- Cost vs accuracy optimization

4.9 Prompt Management

Evaluate

- Prompt templates and reuse
- Versioning of prompts
- Separation from code (config-driven)

Best Practices

- Parameterized prompts
- Central prompt registry
- A/B testing prompts

4.10 Agent Versioning

Requirements

- Version control for agents
- Backward compatibility
- Safe rollout

Evaluate

- Version tagging
- Deployment strategies (canary, staged)
- Rollback capability

4.11 Identity & Access Management

Evaluate

- Authentication for agents and tools
- (Role/Agent Based Access Control) RBAC/ABAC policies
- Secure agent-to-tool communication

Key Questions

- How are agents authenticated?
- How is cross-agent access controlled?

4.12 Guardrails & Policy Management

Guardrails

- Input validation — block or sanitize unsafe, off-topic, or non-compliant user or agent inputs
- Output validation — enforce response quality, format, and safety before results reach the user

- Tool-use constraints — limit which tools an agent may invoke and under what conditions
- PII and sensitive-data handling — detect, redact, or block exposure of regulated or confidential data

Policy Management

- Central policy registry for reusable guardrail and access rules across agents and workflows
- Policy versioning and environment-specific rollout (dev, staging, production)
- Tenant- or role-scoped policies aligned with RBAC/ABAC and subscription tier

Evaluate

- Built-in vs external guardrail services (content safety, moderation, custom validators)
- Declarative policy definition (config-driven rather than hard-coded in agent logic)
- Enforcement points across the workflow (supervisor, agent, tool, and response layers)
- Auditability when a guardrail or policy blocks, modifies, or redirects execution

Key Questions

- Can guardrails and policies be defined and externalized as JSON (or similar) configuration?
- Does the SDK or framework load, validate, and enforce policies at runtime without code changes?
- How are policy violations logged, traced, and surfaced for operators and compliance review?
- Can policies be applied consistently across multi-agent workflows and MCP tool calls?

4.13 Multi-Version Agents (Tiering)

Use Case

- Teaser
- Trailer
- Full-feature

Evaluate

- Routing logic based on subscription tier
- Version selection at runtime
- Feature gating mechanisms

4.14 Multi-Tenancy & Cost Tracking

Requirement

Support:

- Retailer (tenant-level)
- User-level
- Agent-level usage

Evaluate

- Tenant isolation
- Usage metering
- Cost attribution

Metrics to Track

- Cost per tenant
- Cost per agent
- Cost per request
- Token usage

---

5. Success Criteria

Functional

- Accurate execution of use case
- Proper agent delegation
- Minimal context passing

Technical

- Modular architecture
- Tool reuse via MCP layer
- Traceable workflows

Enterprise

- Scales to 100+ agents
- Supports versioning & tenancy
- Strong governance model

---

6. Deliverables

For each hyperscaler:

- Architecture diagram
- Working demo
- Agent registry design
- Tool registry approach
- Observability dashboard
- Cost analysis
- Final recommendation

---

7. Summary

This evaluation aims to identify which hyperscaler best supports:

- End-to-end agentic orchestration
- Enterprise-scale agent & tool management
- Governance (identity, versioning, tenancy)

- Efficient cost and performance optimization